DE Morgan's Theorem

CENG 2112.01

Submitted by

David Dulaney

2344357

Computer Science

University of Houston-Clear Lake

Houston, Texas 77058

Feb 10,2026

Contents

Abstract

An AND gate is a device that requires two inputs based off two options which are 0 and 1, then it will produce one output. If the inputs of this gate are the same (ex. Both inputs are 1), then the output will be the same as the inputs. Also, if one of the inputs are 0, then it will produce a 0. An OR gate is a device that requires two inputs based off two options similar to the AND gate and follows the same rules as far as the output produced will be the same as two identical inputs, but they are opposite in the sense that if one if the inputs is a 1, then it will produce a 1. The important finding in this experiment is that when both inputs are the same for both gates, the output will also be the same, but they are opposite in their requirement of producing a 0 or a 1. Because of this, you need to be careful not to be redundant in your circuits. De morgan's theorem reinforces this by showing how two circuits can produce the same outputs if the theorem is understood correctly. For example, not x or not y produces the same output as not(x and y)

Introduction

To be able to understand the OR and AND gates work. We built a schematic to understand where both gates have to be placed in order for it to gain two inputs and produce an output depending on the combination of inputs we send to them. Then we wrote codes to simulate the gates in a circuit and see how they with four combinations of 0s and 1s in a waveform.

Requirements

The requirement for this lab is to simulate what the outputs of the AND and OR gates are in combination on Multisim, then we used the gates on a bread board in combination with themselves and an inverter in order to produce and confirm our experiment is correct with Multisim. Then we coded in VHDL for simulation to produce the same outcome as Multisim and the bread board to fully confirm out experiment.

Prelab

LAB 1: DeMorgan's Law

Prelab: $(xy)' = \overline{xy} \rightarrow \bar{x}+\bar{y} \rightarrow$ truth table

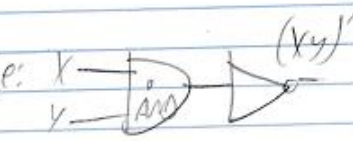
$(x+y)' = \overline{x+y} \rightarrow \bar{x}\bar{y}$ → truth table

x	y	x+y	(xy)'
1	1	0	0
1	0	1	1
0	1	1	1
0	0	1	1

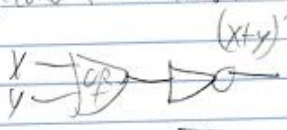
truth table:

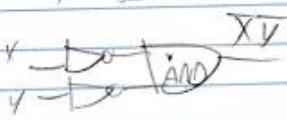
x	y	(x+y)'	$\bar{x}\bar{y}$
1	1	0	0
1	0	0	0
0	1	0	0
0	0	1	1

Circuits for first truth table:



Circuits for second truth table:





Implementation

The first step in implementation was to build a schematic with a word generator and a logic analyzer with 4 combinations of AND and OR gates as well as inverters based on four combinations of 0s and 1s using de morgens theorem as input coming from the word generator then the output going to the analyzer. Then we simulated these combinations in VHDL to see if we get the same outcome as our schematics.

Design Code/Design Diagrams

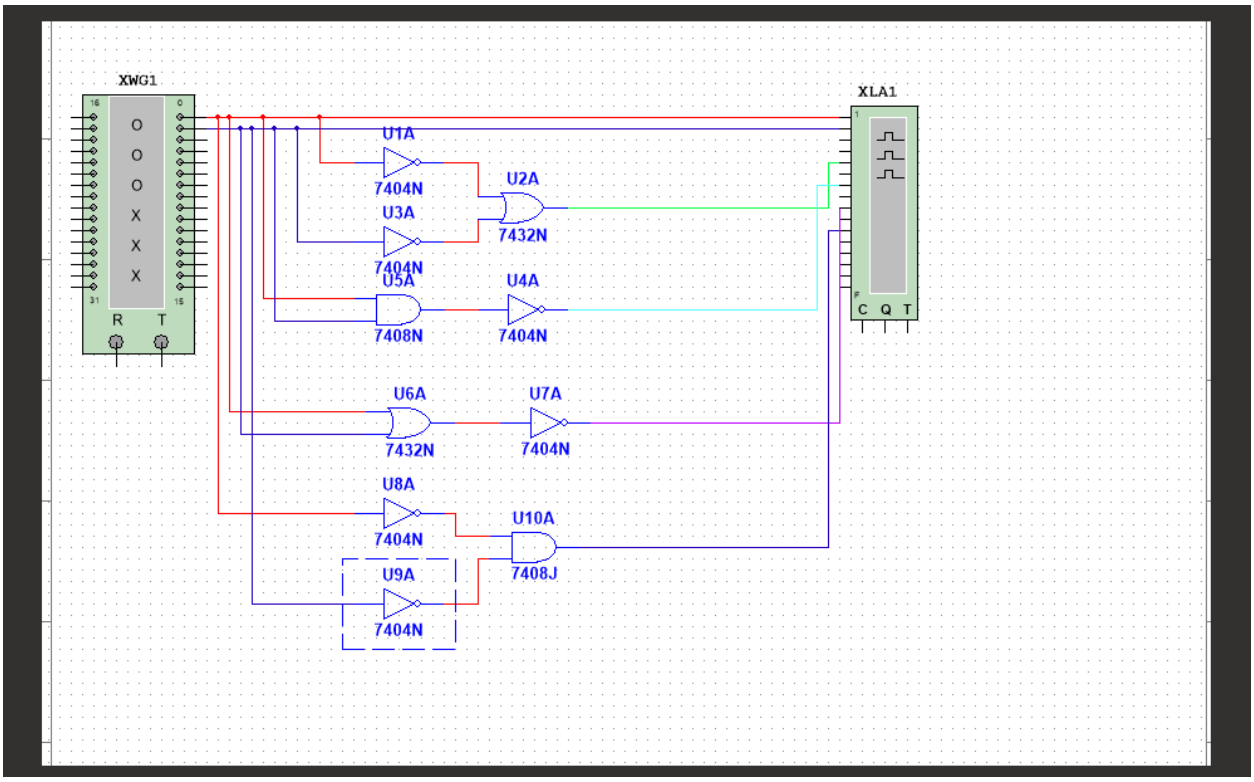


Figure 1: Diagram for four combinations of AND and OR gates as well as inverters using de morgans theorem

---

-- Company:

-- Engineer:

--

-- Create Date: 02/03/2026 02:54:41 PM

```
-- Design Name:
-- Module Name: DM - Behavioral
-- Project Name:
-- Target Devices:
-- Tool Versions:
-- Description:
--
-- Dependencies:
--
-- Revision:
-- Revision 0.01 - File Created
-- Additional Comments:
--
----------------------------------------------------------------------------------


library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
```

```
--use UNISIM.VComponents.all;
--blackbox generation
entity DM is
    Port ( X : in STD_LOGIC;
           Y : in STD_LOGIC;
           L1A : out STD_LOGIC;
           L1B : out STD_LOGIC;
           L2A : out STD_LOGIC;
           L2B : out STD_LOGIC);
end DM;

architecture Behavioral of DM is

begin
-- Inserted logic inside box
L1A <= not(X and Y);
L1B <= not(x) or not(y);
L2A <= not(x or y);
L2B <= not(x) and not(y);

end Behavioral;
```

Schematics

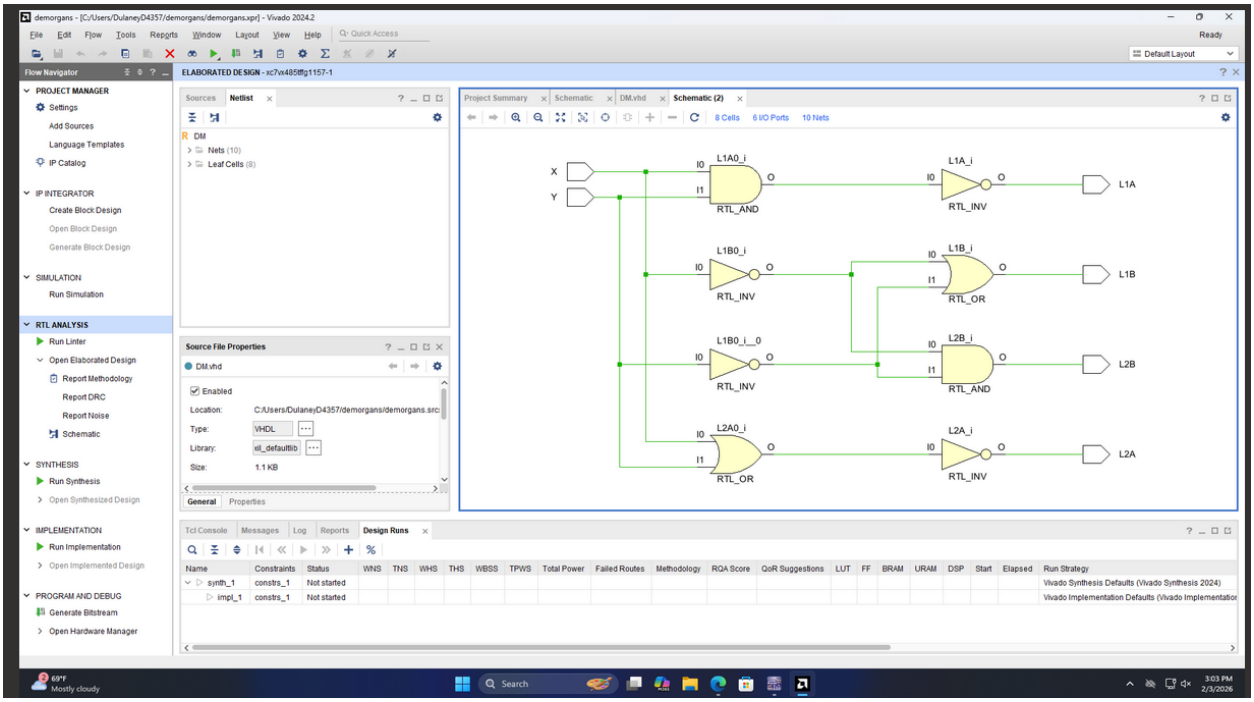


Figure 2: schematic for four combinations of AND and OR gates with inverters based on De morgans theorem

Testbench

```
----------------------------------------------------------------------------------
-- Company:
-- Engineer:
--
-- Create Date: 02/03/2026 03:06:41 PM
-- Design Name:
-- Module Name: TB - Behavioral
-- Project Name:
-- Target Devices:
-- Tool Versions:
-- Description:
--
-- Dependencies:
--
```

```
-- Revision:
-- Revision 0.01 - File Created
-- Additional Comments:
--
----------------------------------------------------------------------------------


library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;
--canvas creation
entity TB is
--  Port ( );
end TB;

architecture Behavioral of TB is
--Place chip on board
component DM is
```

```
    Port ( X : in STD_LOGIC;
           Y : in STD_LOGIC;
           L1A : out STD_LOGIC;
           L1B : out STD_LOGIC;
           L2A : out STD_LOGIC;
           L2B : out STD_LOGIC);
end component;
signal WX, WY, WL1A, WL1B, WL2A, WL2B: std_logic; --instantiating wires for each port.
begin
MappingWires2BoxPorts: DM  port map(X => WX,Y=> WY, L1A => WL1A, L1B => WL1B, L2A =>
WL2A, L2B => WL2B);
process
begin
WX <= '0'; WY <='0'; wait for 10ns;
WX <= '0'; WY <='1'; wait for 10ns;
WX <= '1'; WY <='0'; wait for 10ns;
WX <= '1'; WY <='1'; wait for 10ns;
wait;


end process;



end Behavioral;
```

Waveform/Results

The waveforms in figures 3 and 4 represent four combinations of AND and OR gates with inverters using de morgans theorem being used at once. In figure 3, the red line shows the voltage when both inputs are fluctuating between four combinations of 0s and 1s. Where green and teal produce similar results even though green is representing two inverters with an OR gate while teal is representing an AND gate with an inverter. Similarly, the purple and pink lines produce the same outputs even though the pink line is representing one OR gate and an inverter while the purple line is representing two inverters with an AND gate.

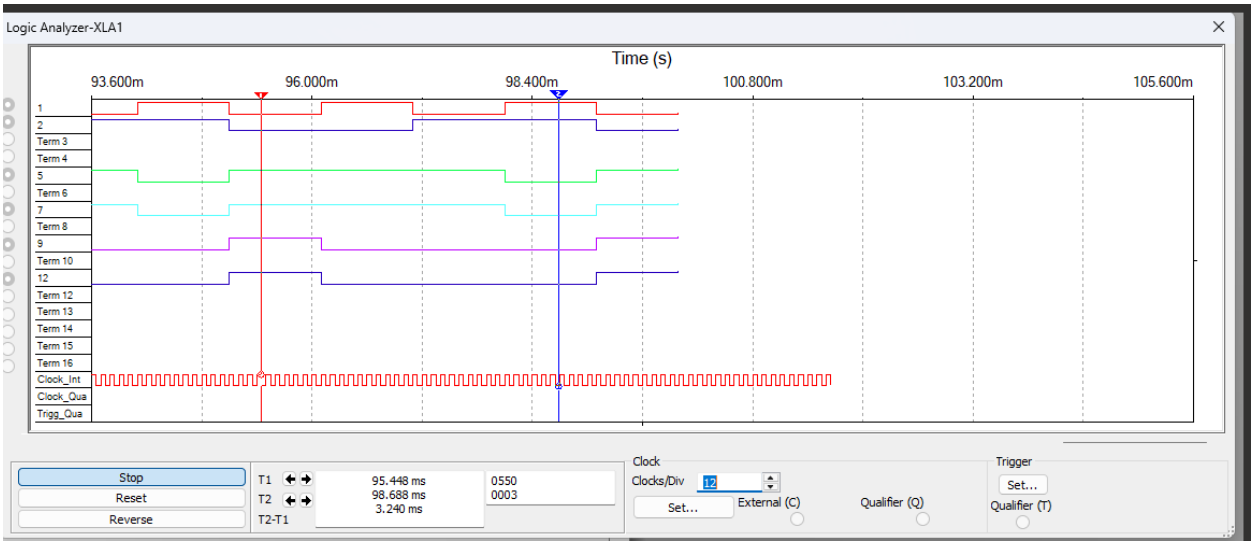


Figure 3: Waveform of four combinations of AND and OR gates with inverters using de morgans theorem

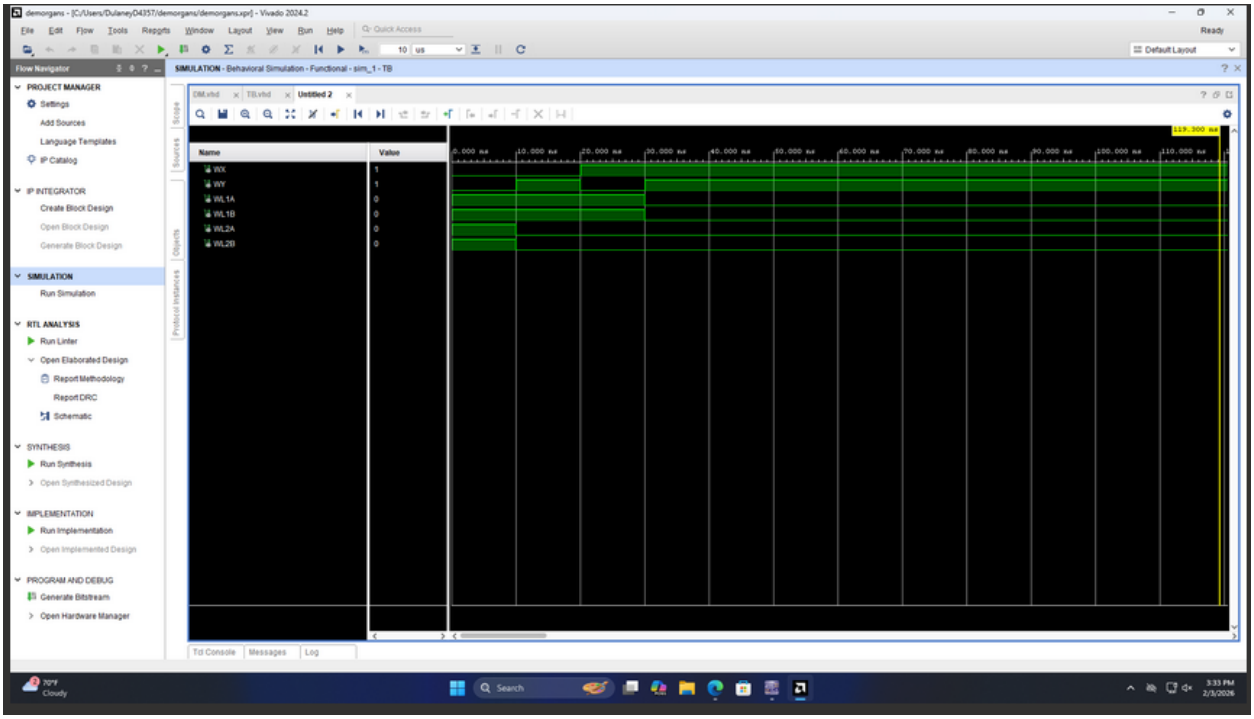


Figure 4: Behavioral simulation of four combinations of AND and OR gates with inverters using de morgans theorem

The result in figure 5 shows the input that the program is submitting is 0 for the least significant bit (y) and a 1 for the bit next to it(x) and the output for both combinations of circuits is 1 (indicated by a green light on led 0 and 1) by using two inverter's outputs and using an OR gate to take those outputs as inputs to produce a 1 as well as using two inputs through an AND gate and then feeding that output into an inverter to produce a 1.
The result in figure 5.2 shows the input that the program is submitting is 1 for the least significant bit (y) and a 1 for the bit next to it(x) and the output for both combinations of circuits is 0 (indicated by no green light on led 0 and 1) by using two inverter's outputs and using an OR gate to take those outputs as inputs to produce a 0 as well as using two inputs through an AND gate and then feeding that output into an inverter to produce a 0.

The result in figure 5.3 shows the input that the program is submitting is 1 for the least significant bit (y) and a 0 for the bit next to it(x) and the output for both combinations of circuits is 1 (indicated by the green light on led 1 and 1) by using two inverter's outputs and using an OR gate to take those outputs as inputs to produce a 1 as well as using two inputs through an AND gate and then feeding that output into an inverter to produce a 1.

The result in figure 5.4 shows the input that the program is submitting is 0 for the least significant bit (y) and a 0 for the bit next to it(x) and the output for both combinations of circuits is 1 (indicated by the green light on led 1 and 1) by using two inverter's outputs and using an OR gate to take those outputs as inputs to produce a 1 as well as using two inputs through an AND gate and then feeding that output into an inverter to produce a 1.

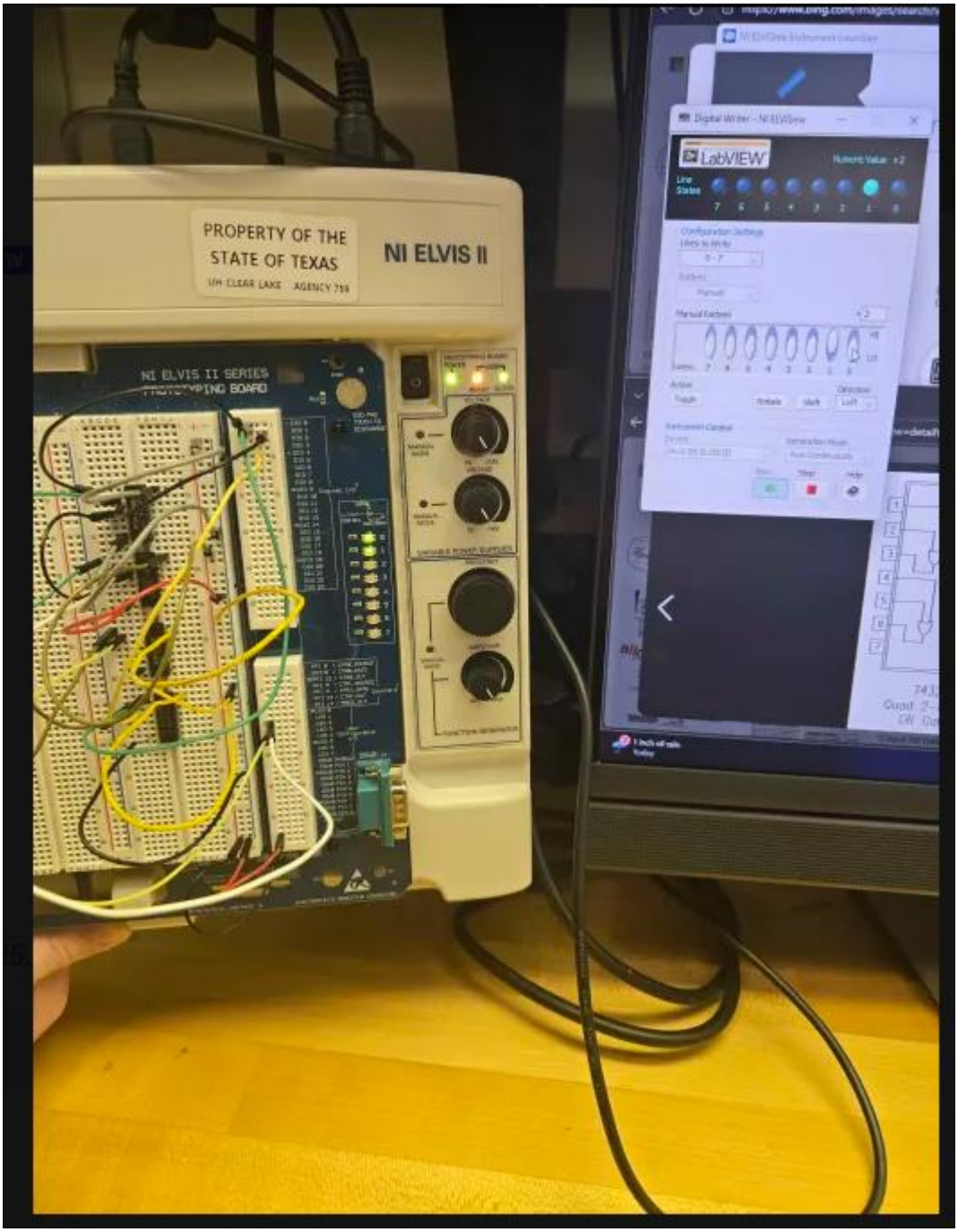


Figure 5: physical board showing a combination of two circuits with an input of the LSB being 0 and the next bit being 1

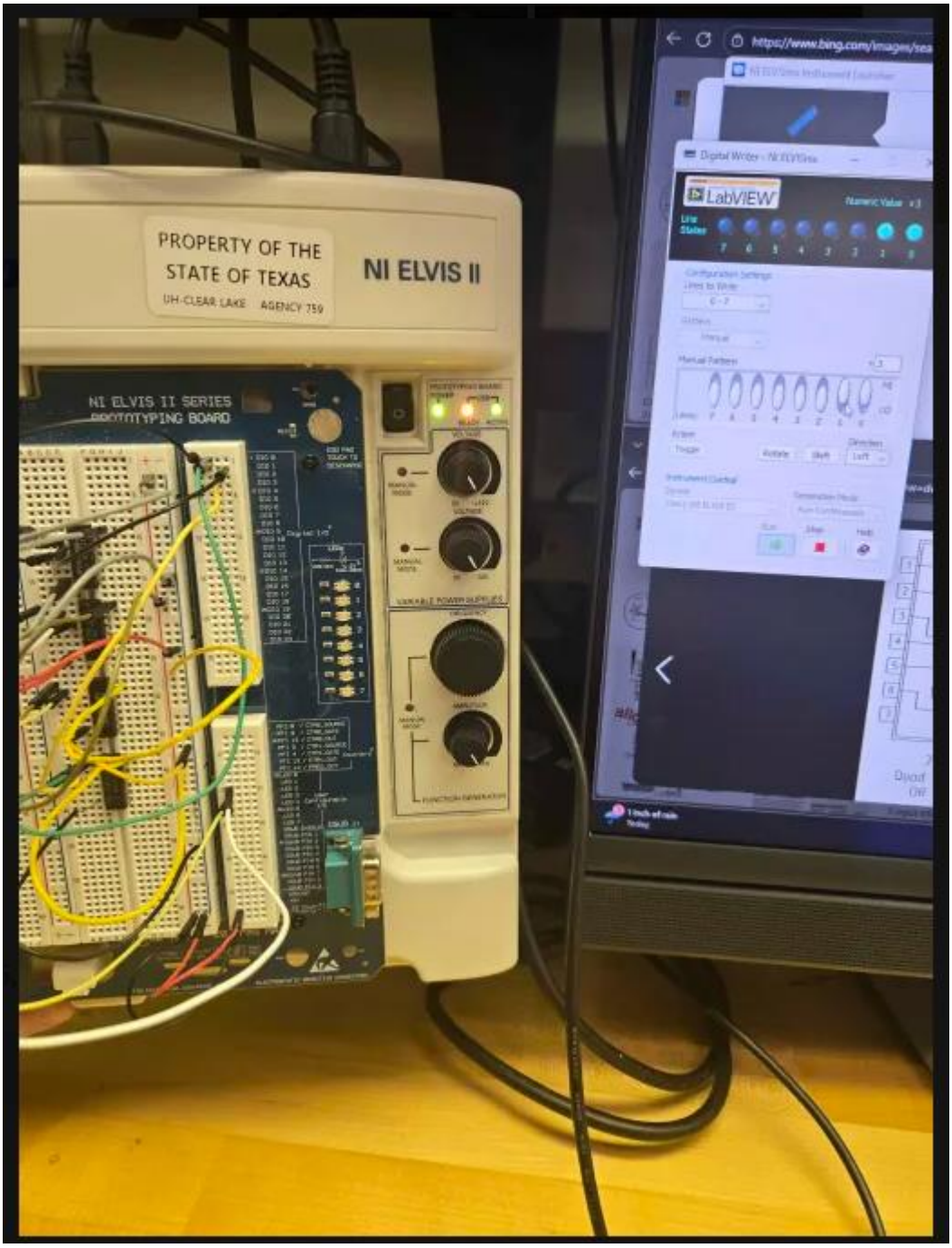


Figure 5.2: physical board showing a combination of two circuits with an input of the LSB being 1 and the next bit being 1

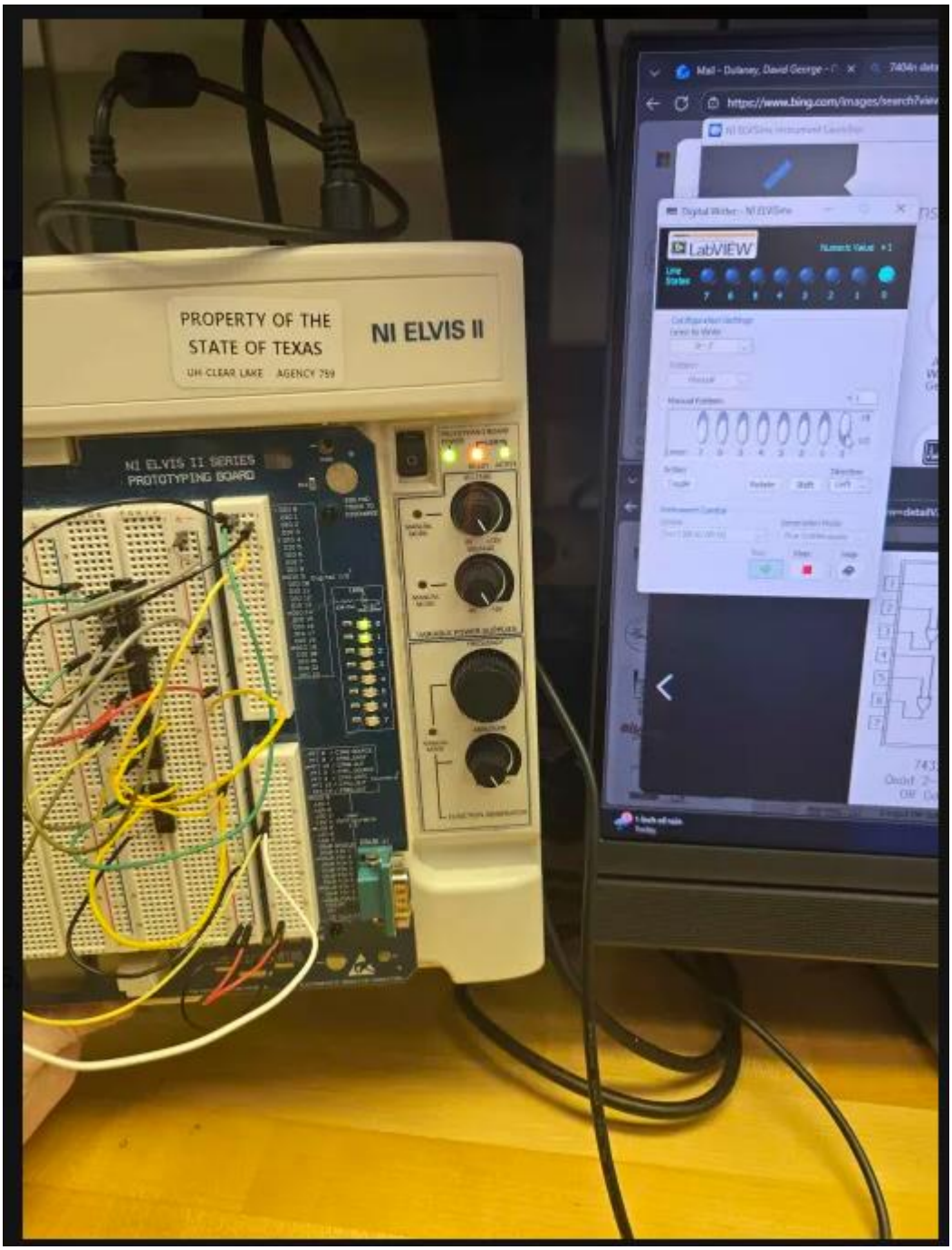


Figure 5.3 physical board showing a combination of two circuits with an input of the LSB being 1 and the next bit being 0

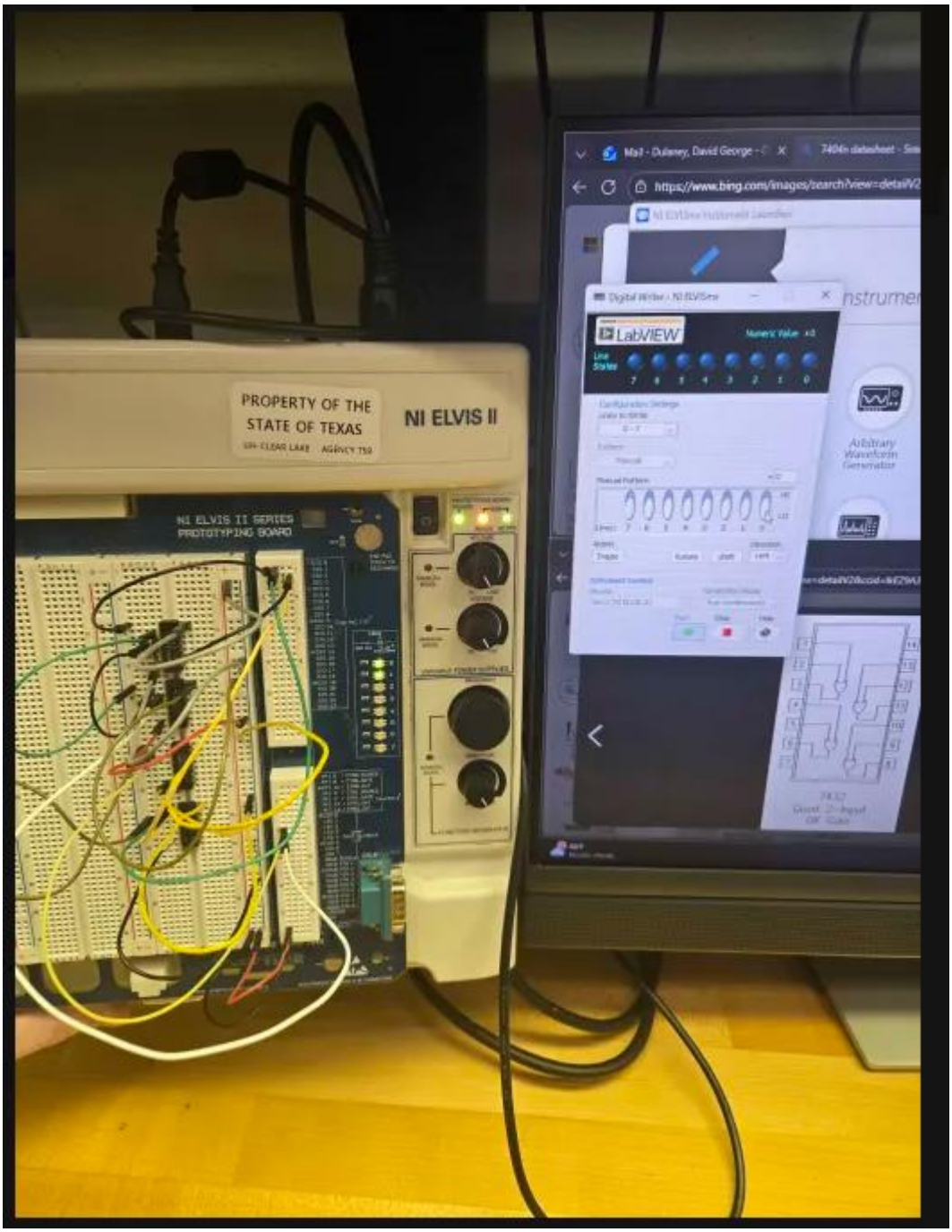


Figure 5.4 physical board showing a combination of two circuits with an input of the LSB being 0 and the next bit being 0

Conclusion

This project was designed to understand that practicing de morgans theorem through a different combination of circuits, you can reach the same output and be more efficient with circuitry involving multiple AND and OR gates along with inverters to produce an output you desire.